

Documento de Alcance - Actualizacion E2

Proyecto: VibeBuilder

Archivo entregable: alcance_e2.pdf

Fuente editable: docs/delivery-2/alcance_e2.md

Inicio de Delivery 2: commit dc52d6c

Corte analizado: HEAD (17ff1ea, Implement robust version regeneration)

Fecha: 23/06/2026

Objetivo de la actualizacion

Actualizar el alcance del proyecto para Delivery 2, incorporando los cambios realizados desde el inicio de la fase y dejando explicitos los nuevos requerimientos funcionales, los requerimientos refinados de Delivery 1, los elementos que pasan a backlog/E3 y las reglas de validacion y excepcion necesarias para que el producto deje de ser solo un flujo de prompt y se comporte como un builder controlable y recuperable.

Delivery 2 mantiene el foco mobile-first del producto: el usuario sigue creando y modificando web apps desde Android mediante prompts, sin editar codigo manualmente. La diferencia principal es que ahora el sistema agrega mayor control sobre proyectos, versiones, artefactos generados, preview, errores y recuperacion ante fallas.

Changelog del alcance

Changelog del alcance

Incluye RF nuevos de E2, RF de E1 refinados y RF que pasan a E3/backlog.

Nuevas User Stories

Se incluyen historias con criterios de aceptacion completos en formato Given/When/Then.

Excepciones y validaciones

Se documentan casos de error, validaciones de negocio y reglas explicitas para evitar inconsistencias.

RF nuevos incorporados en E2

ID	Requerimiento funcional	Descripcion	Evidencia/alcance tecnico
RF-E2-01	Gestion de proyectos y metadatos	El usuario puede editar titulo/descripcion, eliminar proyectos, buscar y ordenar proyectos desde la app.	PATCH /projects/:id, DELETE /projects/:id, busqueda y ordenamiento en Home.
RF-E2-02	Estructura de artefacto generado	Cada version exitosa debe guardar un artefacto de web app con manifiesto, archivos, framework, dependencias y referencia de preview/exportacion.	backend/src/artifacts/, tablas version_artifacts y version_artifact_files, export ZIP.
RF-E2-03	Exportacion de versiones	El usuario propietario puede descargar/exportar una version generada como archivo ZIP.	GET /projects/:projectId/versions/:versionNumber/export.

ID	Requerimiento funcional	Descripción	Evidencia/alcance técnico
RF-E2-04	Validación de salida generada	El backend debe validar estructura, manifiesto y rutas seguras antes de considerar una generación como exitosa.	manifest-validator, contrato de artefactos y estados de validación.
RF-E2-05	Preview mejorada	La app debe resolver previews actuales o históricas, detectar estados no disponibles y ofrecer apertura externa/QR.	GET /projects/:projectId/preview con target=current o target=version, fallback de providerMeta.previewUrl, UI de preview.
RF-E2-06	Historial de versiones mejorado	El historial debe diferenciar versiones exitosas, fallidas y en progreso, mostrar causas de falla y permitir acciones según estado.	Estados QUEUED, GENERATING, VALIDATING, READY, FAILED, CANCELLED; tarjetas de historial con regeneración.
RF-E2-07	Regeneración robusta	Una versión fallida puede regenerarse sin duplicar proyectos ni borrar el intento anterior.	POST /projects/:id/version/:versionId/regenerate, sourceVersionId, attemptNumber, failureCode.
RF-E2-08	Integración v0 configurable por sesión	El usuario puede guardar, probar y eliminar la API key de v0 asociada a su sesión.	Pantalla V0IntegrationScreen, endpoints /integrations/v0 y /integrations/v0/test.
RF-E2-09	Mejor manejo de errores	Los errores deben tener códigos estables, mensajes accionables y separación entre detalle técnico y mensaje de usuario.	Contrato de error con code, message, retryable y estados específicos de preview/generación.
RF-E2-10	Persistencia y despliegue mejorados	El backend debe poder ejecutarse localmente o en entorno desplegado, con configuración para Vercel/Turso y almacenamiento de artefactos.	backend/vercel.json, database-connection.js, .env.example, arquitectura Supabase/Turso documentada.
RF-E2-11	Readiness y pruebas	Delivery 2 debe poder verificarse con tests backend y Android enfocados en metadatos, preview, artefactos, regeneración y errores.	Tests en backend/test/* y pruebas ViewModel/repository en Android.

RF de E1 modificados o refinados

RF original de E1	Cambio en E2	Motivo
Crear proyecto con título y descripción	Se agrega edición posterior, validación de metadatos, soft delete, búsqueda y ordenamiento.	El usuario necesita organizar proyectos reales una vez que empieza a iterar.
Enviar prompt y generar versión	La generación ahora produce versión con estados, posible fallo, artefacto asociado, idempotencia y trazabilidad del proveedor.	Evitar que una respuesta parcial o rota sea tratada como éxito.
Preview funcional	El preview deja de depender solo de currentVersion.previewUrl y puede resolverse desde backend, provider metadata o una versión histórica.	Las URLs pueden expirar o no venir en el mismo campo; la app necesita recuperación accionable.
Historial básico	El historial ahora muestra estado, causa de falla, versión activa, intentos fallidos y acción de regeneración.	El historial pasa de lista informativa a superficie de control del proyecto.
Reintento básico de generación	Se redefine como regeneración robusta sobre versiones fallidas, creando una nueva versión sin sobrescribir la anterior.	Mantener trazabilidad y evitar duplicados por doble tap, timeout o reenvío de red.
Backend mínimo persistente	Se agregan metadatos, artefactos, storage, configuración de despliegue y soporte Turso/Vercel.	Delivery 2 exige confiabilidad y reproducibilidad, no solo flujo feliz.
Errores genéricos	Se refinan con códigos estables: preview no listo, expirado, no disponible, proyecto no encontrado, validación inválida, proveedor con timeout, etc.	Los usuarios necesitan una acción clara y los desarrolladores necesitan diagnóstico.

RF que se postergan a E3 o backlog

ID	Requerimiento postergado	Nueva fase/backlog	Justificacion
RF-BK-01	Biblioteca publica de proyectos	Delivery 3	Requiere modelo de visibilidad, moderacion y busqueda publica.
RF-BK-02	Fork de proyectos de otros usuarios	Delivery 3	Depende de biblioteca publica, ownership y atribucion.
RF-BK-03	Perfil social/comunidad	Delivery 3	No bloquea el objetivo de control y calidad de E2.
RF-BK-04	Publicacion final/marketplace	Backlog posterior	Antes se debe estabilizar preview, export y validacion.
RF-BK-05	Editor visual o editor de codigo integrado	Backlog posterior	Contradice el enfoque prompt-first de las entregas iniciales.
RF-BK-06	Generacion de APK nativo	Fuera de alcance actual	La decision de producto mantiene salida web por facilidad de preview y despliegue.
RF-BK-07	Colaboracion multiusuario en tiempo real	Backlog posterior	Necesita autenticacion/roles y control de concurrencia no requeridos para E2.
RF-BK-08	Restaurar versiones como flujo completo	Backlog E2/E3	El historial y la regeneracion avanzan primero; restore completo puede agregarse cuando el modelo de artefactos este estable.
RF-BK-09	Validacion aislada con build completo y limites de CPU/memoria	Backlog E2 hardening	E2 incorpora validacion estructural; el build aislado completo requiere infraestructura adicional.
RF-BK-10	Paginacion cursor completa del historial	Backlog E2 hardening	El listado actual mantiene limite operativo; la paginacion fina se prioriza cuando haya historiales largos reales.

Nuevas User Stories

US-E2-01 - Editar y organizar proyectos

Como usuario creador,

quiero renombrar, describir, buscar, ordenar y eliminar mis proyectos,

para mantener organizado mi espacio de trabajo cuando tenga varias apps generadas.

Criterios de aceptacion

- **Given** que tengo al menos un proyecto creado, **when** cambio su titulo y descripcion, **then** el nuevo dato aparece en Home y Project Detail despues de recargar.
- **Given** que ingreso un titulo vacio o demasiado largo, **when** intento guardar, **then** la app muestra validacion y no persiste el cambio.
- **Given** que elimino un proyecto, **when** vuelvo al listado, **then** el proyecto ya no aparece y sus endpoints quedan inaccesibles para la sesion normal.
- **Given** que busco por texto, **when** el titulo o descripcion coincide, **then** el proyecto aparece filtrado sin modificar la fuente de datos.

US-E2-02 - Preview confiable de versiones

Como usuario creador,

quiero abrir el preview de la version actual o de una version historica,

para revisar el resultado generado antes de seguir iterando.

Criterios de aceptacion

- **Given** que existe una version exitosa con preview disponible, **when** abro la pestana Preview, **then** la app resuelve una URL valida y permite abrirla externamente.
- **Given** que una version historica tiene preview, **when** la selecciono desde Historial, **then** puedo abrir esa version sin cambiar la version activa del proyecto.
- **Given** que el preview esta expirado, no listo o no disponible, **when** intento abrirlo, **then** recibo un mensaje especifico y una accion de recuperacion cuando corresponda.
- **Given** que la URL se resuelve desde `providerMeta.previewUrl`, **when** no existe `previewUrl` directo, **then** la app igualmente puede abrir el preview.

US-E2-03 - Regenerar una version fallida

Como usuario creador,

quiero regenerar una version fallida dentro del mismo proyecto,

para recuperarme de errores sin perder historial ni crear proyectos duplicados.

Criterios de aceptacion

- **Given** que una version esta en estado fallido, **when** toco Regenerar, **then** se crea una nueva version con referencia a la version fuente.
- **Given** que toco varias veces o se repite la solicitud de red, **when** se usa la misma clave de idempotencia, **then** no se crean versiones duplicadas.

-
- **Given** que existia una version exitosa previa, **when** la regeneracion falla, **then** la version exitosa previa sigue siendo la activa.
 - **Given** que la regeneracion tiene exito, **when** termina el proceso, **then** el historial muestra el nuevo intento y el proyecto apunta a la version exitosa nueva.

US-E2-04 - Guardar artefactos generados

Como usuario creador,

quiero que cada version exitosa guarde sus archivos y metadatos,

para poder exportar, validar y reproducir la app generada aunque cambie la URL temporal del proveedor.

Criterios de aceptacion

- **Given** que una generacion termina correctamente, **when** se guarda la version, **then** existe un artefacto inmutable asociado.
- **Given** que el artefacto contiene rutas inseguras, archivos duplicados o manifiesto invalido, **when** se valida, **then** la version se marca como fallida y no reemplaza la version activa.
- **Given** que soy propietario del proyecto, **when** solicito exportar una version, **then** recibo un ZIP con archivos validos.
- **Given** que no soy propietario, **when** intento acceder al artefacto o export, **then** el sistema responde como no encontrado/no autorizado sin filtrar informacion.

US-E2-05 - Gestionar integracion v0

Como gestor tecnico del sistema o usuario que configura su sesion,

quiero guardar, probar y eliminar mi clave de v0,

para controlar la conexion con el proveedor de generacion sin exponer secretos en la app.

Criterios de aceptacion

- **Given** que ingreso una API key, **when** la guardo, **then** el backend la conserva asociada a mi sesion y la app muestra confirmacion.
- **Given** que pruebo la conexion, **when** v0 responde correctamente, **then** la app informa conexion exitosa.
- **Given** que la clave es invalida o el proveedor falla, **when** pruebo la conexion, **then** recibo un error accionable sin revelar secretos.
- **Given** que elimino la clave, **when** vuelvo a consultar el estado, **then** la sesion ya no figura configurada.

Gestion de excepciones y validaciones

Casos de error nuevos o reforzados en E2

Caso	Comportamiento esperado
Proyecto inexistente, eliminado o ajeno a la sesion	Responder 404 o equivalente sin revelar si pertenece a otro usuario.
Metadatos invalidos	Bloquear persistencia y mostrar validacion local/remota.
Doble envio de prompt o regeneracion	Usar idempotencia para evitar versiones duplicadas.
Regeneracion sobre version no fallida	Rechazar con error estable: solo se pueden regenerar versiones fallidas.
Error/timeout del proveedor v0	Crear o mantener version fallida con <code>failureCode</code> y mensaje de recuperacion.
API key de v0 ausente o invalida	Informar configuracion requerida o conexion fallida sin exponer la clave.
Manifiesto invalido	Marcar version como fallida; no actualizar version activa.
Ruta insegura o path traversal	Rechazar archivo/artefacto y registrar error de validacion.
Archivo duplicado, demasiado grande o tipo no soportado	Rechazar artefacto o marcar validacion fallida.
Falla de storage al guardar artefacto	No marcar version como exitosa.
Preview no listo	Mostrar estado accionable y permitir reintento/refresh.
Preview expirado	Informar expiracion y resolver nuevamente si el proveedor lo permite.
Preview no disponible	Mantener historial y mostrar alternativa de recuperacion.
Error de red u offline en Android	Mostrar error especifico y permitir reintentar sin perder draft.
Export de version ajena o inexistente	Responder 404/no autorizado y no filtrar archivos.

Reglas de negocio explicitas

1. Un proyecto pertenece a una sesion/usuario; otra sesion no puede editarlo, eliminarlo, exportarlo ni ver sus versiones.
2. Un proyecto eliminado por soft delete no aparece en listados normales y bloquea acciones posteriores.
3. Una version exitosa debe tener estado final, metadatos consistentes y, cuando aplique, artefacto asociado.
4. Los artefactos de versiones finales son inmutables.
5. Una version fallida queda en historial; no se borra ni se sobrescribe.
6. Regenerar siempre crea una version nueva dentro del mismo proyecto.
7. La version activa solo cambia cuando una nueva generacion/regeneracion termina exitosamente.
8. Las rutas de archivos generados deben ser relativas, seguras y sin traversal.
9. La app generada sigue siendo web app, preferentemente React + Vite + TypeScript.
10. La app Android no muestra stack traces, secretos ni logs internos al usuario final.
11. Las claves/API secrets no se devuelven al cliente ni se registran en logs.
12. El preview puede abrirse en navegador externo, pero enlaces no confiables no deben ejecutarse como parte del control interno de la app.
13. Las operaciones con posibilidad de repeticion de red deben ser idempotentes.
14. Las fallas de validacion no son reintentos automaticos; requieren regeneracion o correccion.

Definition of Done de Delivery 2

Delivery 2 se considera completo cuando un usuario puede:

1. Crear, editar, buscar, ordenar y eliminar proyectos desde Android.
2. Enviar prompts y obtener versiones con estados claros.
3. Guardar artefactos de versiones exitosas y exportarlos cuando corresponda.
4. Ver y abrir preview de la version actual o historica cuando este disponible.
5. Ver historial con versiones exitosas y fallidas.
6. Regenerar una version fallida sin duplicar proyecto ni perder trazabilidad.
7. Recibir errores accionables ante preview no disponible, proveedor fallido, validacion invalida o red inestable.
8. Ejecutar pruebas backend y Android relevantes desde un checkout limpio.

Los flujos de comunidad, forks y exploracion publica quedan fuera de este cierre y pasan al alcance de Delivery 3.